# Worksheet: Principal Component Analysis (PCA) of PFAS Concentrations

This worksheet will guide you through loading, cleaning and analyzing PFAS data with PCA in **Python (Jupyter)** or **R**.

## Load, Inspect and Prepare Data

1. Load and inspect the dataset
    a. Load the dataset into Python or R.
    b. List all PFAS compounds included in the dataset.
    c. List all samples identifiers included in the dataset (e.g. sample names, locations, dates, water types).

2. Remove PFAS columns with insufficient data
    a. For PCA, each PFAS variable must have enough valid numeric observations to be statistically meaningful.
    b. Decide on a **criterion** for excluding PFAS compounds with too many missing or MDL values.
    c. Apply your criterion and **remove low-availability PFAS columns**.
    d. Report **how many PFAS columns remain** after cleaning.

3. Handle Remaining MDL values.
    a. Identify any remaining MDL values in the dataset.
    b. Develop a strategy to replace them with reasonable numeric concentration values. (e.g. substitute with **½ MDL**, Substitute with **MDL / $\sqrt{2}$**, or substitute with 0)
4. Check and Convert Data Types
    a. Determine the data type of each column (numeric, string, categorical, etc.).
    b. Decide how to handle non-numeric columns:
    c. **Python:**
        i. PCA requires *all columns to be numeric*.
        ii. Convert or remove non-numeric columns before PCA.
    d. **R:**
        i. PCA requires numeric concentration columns, but non-numeric columns can be kept as supplementary qualitative variables. This is done by setting the quali.sup argument in FactoMineR::PCA().
        ii. Convert non-numeric concentration columns before PCA.

5. Scale the Data Before PCA
    a. PCA is influenced by variable scale. PFAS compounds vary in their concentration ranges, so scaling is essential.
    b. You will scale each PFAS column using:

$$\text{scaled value} = \frac{\text{value} - \text{column mean}}{\text{column standard deviation}}$$

    c. **Python:** Use StandardScaler from sklearn.preprocessing to scale your dataframe before PCA.

d. **R:** Later in this exercise, you will set the scale = TRUE parameter in FactoMineR::PCA() to standardize the data automatically.

<u>Hints for Python:</u>

- Read in the data with **pandas df = pd.read_csv(*filename*)**
- Explore the number of columns and rows of a dataframe df with **df.shape** or **len(df)**
- Access columns of a dataframe df with **df.columns**
- Access the values of a column of a dataframe df with **df[*column_name*]**
- Delete columns of a dataframe df with **df=df.drop(columns=*["column_name1", "column_name2", "column_name3",...*]**
- Replace values in a dataframe with **df=df.replace(value_to_be_replaced, replacement_value)**
- Import the module **StandardScaler** from **sklearn.deposition** and scale the dataframe df.

<u>Hints for R:</u>

- Read in the data with **df <- read.csv(*filename*)**
- Explore the number of columns and rows of a dataframe df with **nrow(df)** or **ncol(df)**
- Access columns of a dataframe df with **colnames(df)**
- Access the values of a column of a dataframe df with **df$*column_name***
- Delete columns of a dataframe df with **df[, c*("column_name1", "column_name2", "column_name3",...)*] <- list(NULL)**
- Replace values in a dataframe with **df[df == value_to_be_replaced] <- replacement_value**
- Convert columns to numeric type by **df[c*("column_name1", "column_name2", "column_name3",...)*] <- lapply(df[c*("column_name1", "column_name2", "column_name3",...)*], as.numeric)**

## Perform principal component analysis (PCA)

1. Perform principal component analysis (PCA) on your fully cleaned, numeric, and scaled dataset.
    a. Save the PCA output under a **new variable name** (e.g., pca or res.pca).
    b. This PCA object will contain:
        i. scores (sample coordinates in PC space)
        ii. loadings (PFAS contributions to each PC)
        iii. variance explained
2. Describe what the PCA results look like and identify the data types of the key PCA outputs:
    a. scores (sample coordinates in PC space)
    b. loadings (contributions of PFAS variables)
    c. variance explained

3. Determine the number of principal components to retain and review the variance explained by each principal component
4. Create a PC1–PC2 Scores Plot (Python)
    a. Generate a scatter plot with:
        i. PC1 on the x-axis
        ii. PC2 on the y-axis
        iii. Points colored by water type (e.g., groundwater, river, estuary)
    b. Label axes clearly and include a legend.
5. Interpret Sample Group Separation based on the PC1–PC2 plot
6. Identify PFAS with Strongest Loadings
    a. Examine the loadings (variable contributions) for PC1 and PC2.
    b. Identify the PFAS with the largest positive loading on each component.

<u>Hints for Python:</u>

- Import the module from **sklearn.decomposition** and initialize **pca = PCA(*n_components=n*)** with n principal components
- Call **scores = pca.fit_transform(df_scaled)** to perform PCA on the cleaned and normalized data frame **df_scaled**.
- Convert the **scores** to a pandas (pd)dataframe.
- Access the explained variability by **pca.explained_variance_ratio_**
- Access loadings (PFAS expressed in principal components) by pca.components_
- Use the matplotlib.pyplot (plt) module to plot the results

<u>Hints for R:</u>

- Import the libraries **FactoMine**R, and **factoextra**
- Call **res.pca** = **FactoMineR::PCA(df, ncp=n)** to perform PCA on the data frame df, with n principal components.
- Set optional function variables **quali.sup, graph** and **scale** from **FactoMineR::PCA()** to tailor PCA analysis to your needs.
- Generate beautiful plots with **factoextra::fviz_pca_ind(res.pca, geom = 'point', habillage = 'Site')**
- Access loadings (PFAS expressed in principal components) by **res.pca$var$coord**

Beforehand: example data set + programming language + PCA.